

Online Exam Portal

Software Architecture & Design Specification

Software Engineering Mini-Project — Deliverables Part-1

Document ID: OEP-SADS-001

Version 1.0 | Date: 28 September 2026

Prepared by: [Student Name(MANOJ A C, MANOJ GAONKAR, RUTHIK)]

| Roll No.: [PES1UG25CS826, PES1UG25CS827, PES1UG25CS837]

Course: Software Engineering | Institution: [College Name]

Prepared in accordance with IEEE Std 1016-2009 and ISO/IEC/IEEE 42010

Revision History

Version	Date	Author	Description
1.0	28-Sep-2026	[Student Name(s)]	Initial release of the architecture and design specification, derived from OEP-SRS-001.

Table of Contents

1. Introduction

2. Architecture

- 2.1 Architectural Drivers
- 2.2 Architecture Pattern
- 2.3 Component Diagram
- 2.4 Component Description
- 2.5 Technology Stack and Deployment
- 2.6 Traceability to Requirements
- 2.7 Security Architecture

3. Detailed Design

- 3.1 Data Design
- 3.2 UML Sequence Diagrams
- 3.3 API Design
- 3.4 Error Handling
- 3.5 User Interface Design
- 3.6 Design Decisions and Constraints

1. Introduction

1.1 Purpose

This document describes the software architecture and the detailed design of the Online Exam Portal (OEP). It explains how the requirements of OEP-SRS-001 are realised by a set of components, their interfaces and their interactions, and it provides the basis for implementation and for the test cases of OEP-TP-001.

1.2 Scope

The document covers the logical architecture (components, layers, pattern), the security architecture, the data model, sequence diagrams for the main scenarios, the REST API design and the error-handling strategy of release 1.0.

1.3 Definitions and References

Term / Document	Meaning
SADS	Software Architecture & Design Specification (this document).
REST	Representational State Transfer; an HTTP-based API style.
DTO	Data Transfer Object exchanged through the API.
OEP-SRS-001	Software Requirements Specification v1.0.
OEP-TP-001	Test Plan v1.0.
IEEE 1016-2009	Standard for Software Design Descriptions.

1.4 Document Overview

Section 2 presents the architecture: drivers, pattern, component diagram, component responsibilities, technology stack, traceability to requirements and the security architecture. Section 3 presents the design: data model, UML sequence diagrams, API design, error handling, user-interface overview and design decisions.

2. Architecture

2.1 Architectural Drivers

Driver	Source	Architectural consequence
Handle 500–1,000 concurrent exam takers	NFR-01, NFR-04	Stateless application services that scale horizontally behind a load balancer; Redis for shared state
No loss of answers, resume after interruption	NFR-07, FR-13	Server-side autosave of answers in the database; client re-fetches attempt state
Exam fairness and protection of answer keys	SR-07, SR-09	Server-authoritative timer; answer keys stay in the server; separate Evaluation Service
Strong access control and traceability	SR-03, SR-08	Central security filter (JWT + RBAC) and dedicated Audit Service
Maintainability and testability	NFR-08	Modular services with clear interfaces; layered code structure

2.2 Architecture Pattern

The OEP uses a **three-tier layered client–server architecture** with a **RESTful, modular service design**:

- **Presentation layer:** a React single-page application that only talks to the API over HTTPS.

- **Application layer:** a Spring Boot application organised as independent service modules (Authentication, User, Exam, Question Bank, Attempt, Evaluation, Result & Report, Audit) behind an API gateway/security filter. Inside each module the code follows Controller → Service → Repository layering.
- **Data layer:** PostgreSQL for persistent data and Redis for token blacklist, rate-limit counters and session-related caches.

Rationale: layering separates concerns and makes each layer replaceable and testable; a modular (“modular monolith”) service design keeps deployment simple for a student project while allowing the busiest modules (Attempt, Evaluation) to be extracted into separate services later; stateless services satisfy the scalability driver. Alternatives considered: a full microservice architecture (rejected — operational overhead too high for the team size) and a server-rendered monolith (rejected — weaker support for a rich timed-exam UI and autosave).

2.3 Component Diagram

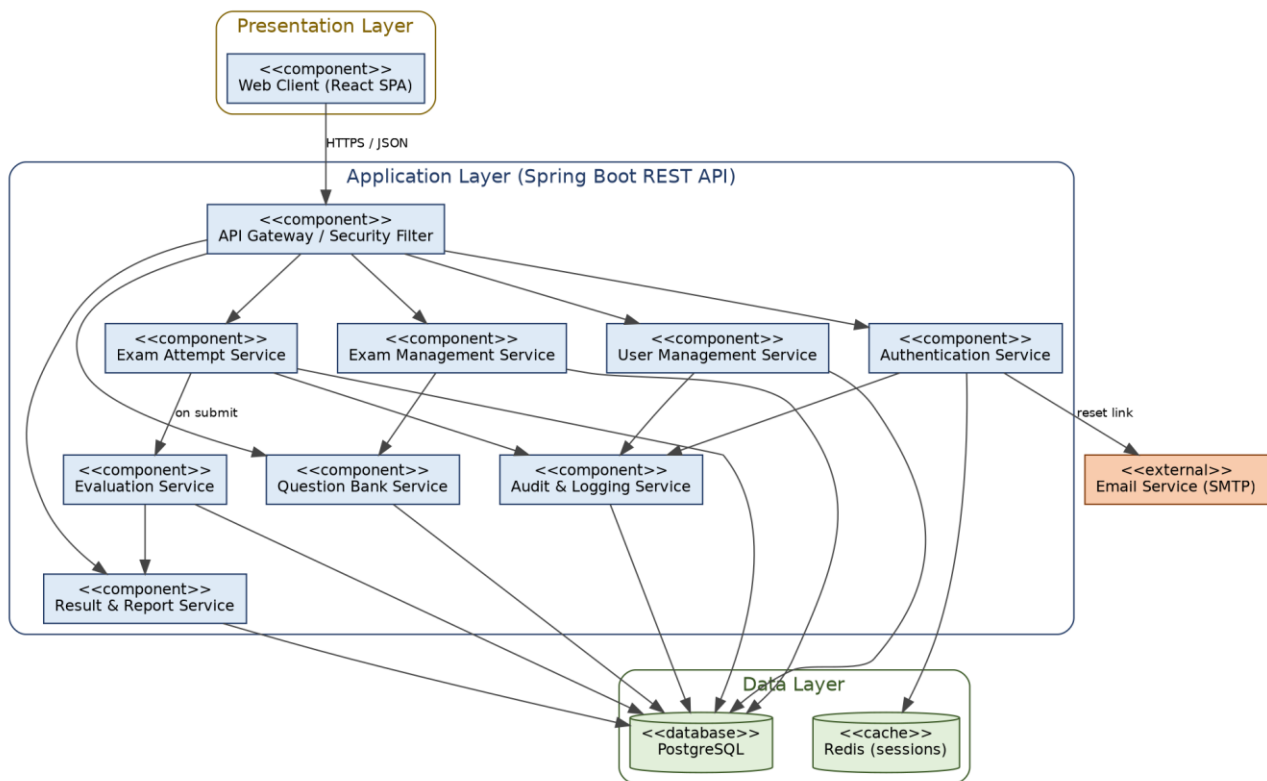


Figure 1: UML component diagram of the Online Exam Portal

2.4 Component Description

Component	Responsibility	Provided interface	Uses
Web Client (React SPA)	Renders role-specific screens, exam timer display, local navigation; never holds answer keys.	Browser UI	API Gateway (HTTPS/JSON)
API Gateway / Security Filter	Terminates TLS, validates JWT, enforces RBAC and rate limits, routes requests, adds correlation ID.	/api/v1/*	Authentication Service, Redis
Authentication Service	Registration, login, logout, token issue/refresh/blacklist, password reset.	/auth/*	PostgreSQL, Redis, SMTP, Audit
User Management Service	Admin CRUD of users and roles, student groups.	/users/*, /groups/*	PostgreSQL, Audit
Exam Management Service	Create/update/publish exams, assign to groups, configure marking and randomization.	/exams/*	PostgreSQL, Question Bank

Component	Responsibility	Provided interface	Uses
Question Bank Service	CRUD and search of questions and options.	/questions/*	PostgreSQL
Exam Attempt Service	Eligibility check, attempt creation, server timer, autosave, submission, resume.	/exams/{id}/attempts, /attempts/*	PostgreSQL, Evaluation, Audit
Evaluation Service	Computes score from stored answers and answer key; applies negative marking.	Internal: evaluate(attemptId)	PostgreSQL
Result & Report Service	Stores and serves results; statistics and CSV export.	/results/*, /reports/*	PostgreSQL
Audit & Logging Service	Append-only recording of security events; admin query.	/audit/* (admin), internal log()	PostgreSQL
PostgreSQL / Redis	Persistent relational storage / in-memory cache and counters.	JDBC / Redis protocol	—

2.5 Technology Stack and Deployment

Concern	Choice
Client	React 18, TypeScript, responsive CSS (360 px and up)
Server	Java 17, Spring Boot 3, Spring Security, Spring Data JPA
Database / cache	PostgreSQL 15 / Redis 7
Build & CI	Maven, JUnit 5, GitHub Actions
Deployment	Docker containers; reverse proxy/load balancer (Nginx) with TLS; two or more application instances

Deployment view: browser → Nginx (TLS termination, load balancing) → N stateless Spring Boot instances → PostgreSQL primary (daily backup, NFR-09) and Redis.

2.6 Traceability to Requirements

Component	Functional requirements	NFR / Security requirements
Web Client	FR-10, FR-12 (display), FR-14, FR-17 (display)	NFR-05, NFR-06, NFR-10
API Gateway / Security Filter	—	SR-02, SR-03, SR-04, SR-06, NFR-01, NFR-04
Authentication Service	FR-01, FR-02, FR-03, FR-04	SR-01, SR-04, SR-06
User Management Service	FR-05	SR-03
Exam Management Service	FR-06, FR-08, FR-09, FR-20	SR-03
Question Bank Service	FR-07	SR-05, SR-07
Exam Attempt Service	FR-10, FR-11, FR-12, FR-13, FR-15	NFR-07, SR-07, SR-09
Evaluation Service	FR-16	NFR-02, SR-07
Result & Report Service	FR-17, FR-18, FR-19	NFR-02
Audit & Logging Service	FR-21	SR-08
PostgreSQL / Redis / Backup	—	NFR-03, NFR-09, SR-01
All modules (engineering practice)	—	NFR-08, SR-05, SR-10

2.7 Security Architecture

Security is designed in layers (defence in depth) and maps to the security objectives SO-1 to SO-5 and requirements SR-01 to SR-10 of the SRS.

Layer	Mechanism	Requirement
Network / transport	HTTPS only (TLS 1.2+), HTTP→HTTPS redirect, HSTS header at Nginx	SR-02
Perimeter	API Gateway rate limiting (5 login requests/min/IP), request size limits, CORS whitelist	SR-06
Authentication	Email + password; bcrypt (cost ≥ 12); account lockout after 5 failures; JWT access token (30 min) with blacklist on logout in Redis	SR-01, SR-04, FR-03
Authorization	Role-based access control checked in the Security Filter and again in each service method (@PreAuthorize); object-level check that an attempt belongs to the caller	SR-03
Application	Server-side validation of all DTOs, parameterized queries via JPA, output encoding and Content-Security-Policy in the client, no answer keys in question DTOs	SR-05, SR-07
Exam integrity	Server-authoritative timer; answers after end time + 10 s grace rejected; randomization performed on the server	SR-09, FR-20
Data	Passwords hashed; database access with least-privilege account; encrypted backups	SR-01, NFR-09
Accountability	Audit Service records login/logout/start/submit/admin actions in an append-only table	SR-08, FR-21
Assurance	OWASP ZAP and dependency scans in CI before release	SR-10

Threat model (STRIDE summary)

Threat	Example	Mitigation
Spoofing	Attacker guesses a password or reuses a stolen token	Lockout, rate limiting, bcrypt, short-lived JWT, blacklist on logout
Tampering	Student edits request to change marks or answers after submission	Server-side evaluation; submitted attempts read-only; input validation
Repudiation	User denies having submitted or changed something	Audit log with user, time, IP
Information disclosure	Answer key or another student's result is read	No key in payload; object-level authorization; TLS
Denial of service	Request flooding during an exam	Rate limiting, stateless scaling, timeouts
Elevation of privilege	Student calls teacher/admin API	RBAC at gateway and service level (HTTP 403)

3. Detailed Design

3.1 Data Design

The main entities and their relationships are summarised below. All primary keys are UUIDs; timestamps are stored in UTC.

Entity	Key attributes	Relationships
User	id, name, email (unique), password_hash, role (STUDENT/TEACHER/ADMIN), status, failed_attempts, locked_until	belongs to many Groups
StudentGroup	id, name	has many Users; assigned to many Exams
Question	id, teacher_id, type (MCQ/MULTI/TF), text, marks, correct_options	has many Options; used in many Exams
Option	id, question_id, text, is_correct	belongs to Question
Exam	id, title, subject, duration_min, start_at, end_at, total_marks, pass_marks, negative_pct, randomize, attempt_limit, status (DRAFT/PUBLISHED)	has many ExamQuestions; assigned to Groups
ExamQuestion	exam_id, question_id, position	joins Exam and Question
Attempt	id, exam_id, student_id, started_at, ends_at, submitted_at, status (IN_PROGRESS/SUBMITTED)	has many Answers; has one Result
Answer	attempt_id, question_id, selected_options, marked_for_review, saved_at	belongs to Attempt
Result	attempt_id, score, percentage, passed, evaluated_at	belongs to Attempt

Entity	Key attributes	Relationships
AuditLog	id, user_id, action, timestamp, ip_address, details (append-only)	references User

3.2 UML Sequence Diagrams

3.2.1 SD-01 User Login

Realises FR-02, FR-03, FR-21, SR-01 and SR-04. The Auth Service verifies the bcrypt hash, records the outcome in the Audit Service and returns a JWT together with the user's role.

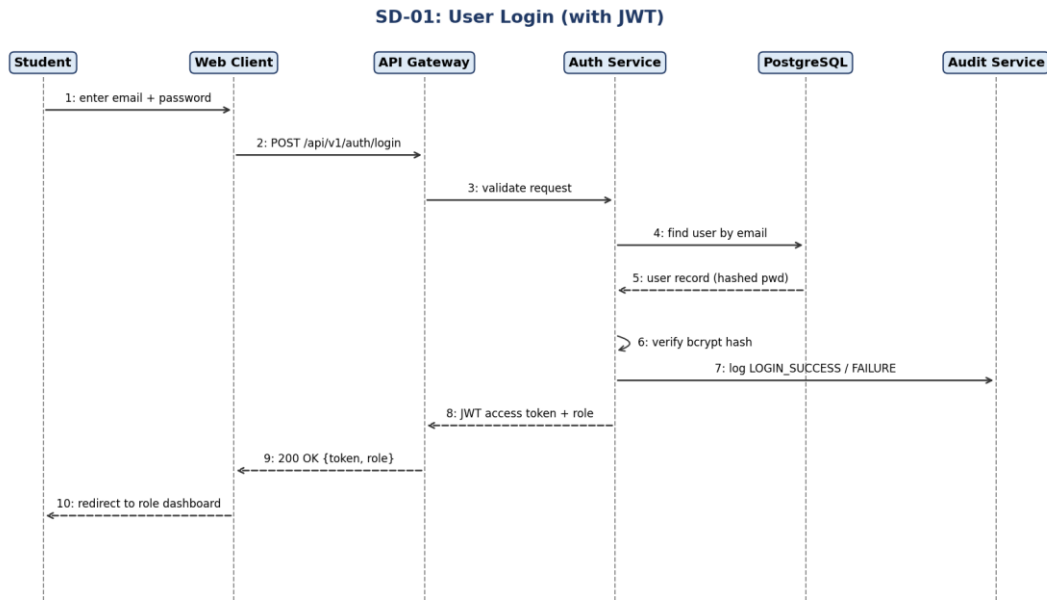


Figure 2: Sequence diagram — User Login

3.2.2 SD-02 Attempt and Submit Exam

Realises FR-11 to FR-16, NFR-07, SR-07 and SR-09. The Attempt Service checks eligibility, starts the server timer, autosaves answers and, on submission, calls the Evaluation Service to compute the score.

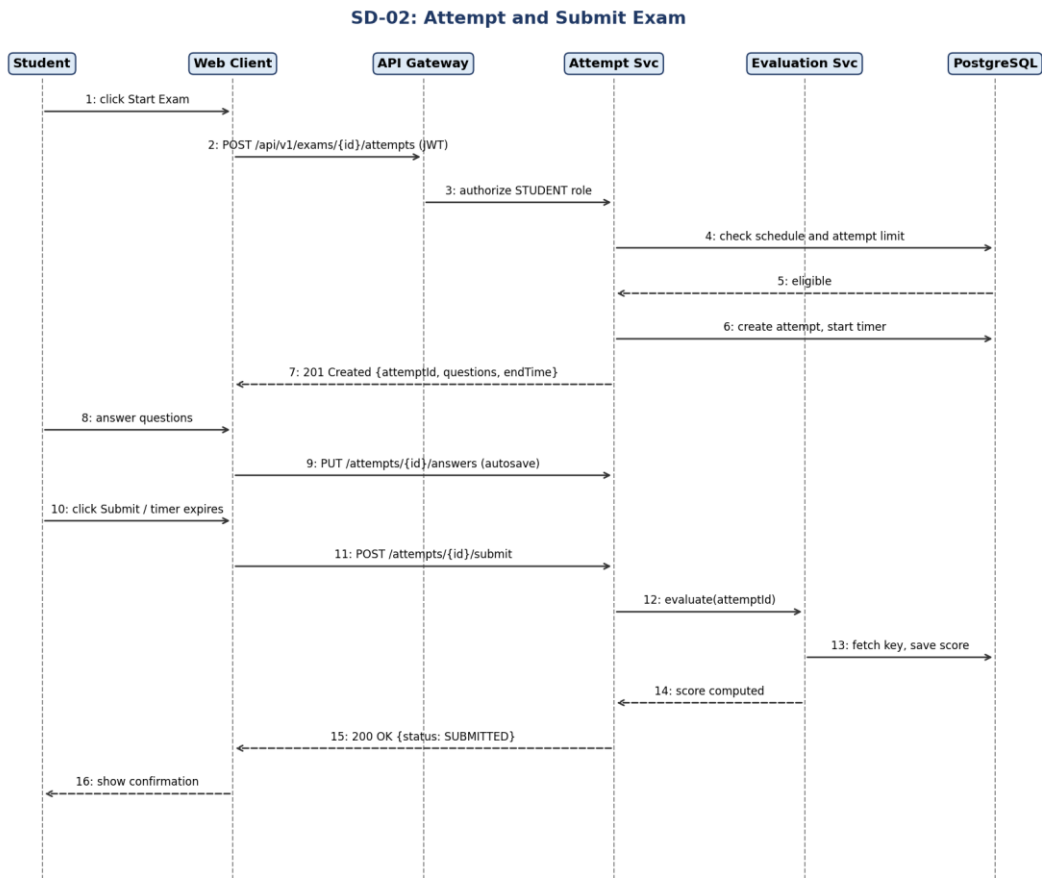


Figure 3: Sequence diagram — Attempt and Submit Exam

3.3 API Design

3.3.1 API Conventions

- Base URL: `https://<host>/api/v1`; JSON request and response bodies (UTF-8); dates in ISO-8601 UTC.
- Authentication: header “Authorization: Bearer <JWT>” on every endpoint except register, login and password-reset.
- Standard status codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests, 500 Internal Server Error.
- Lists support `?page=&size=` (default 0/20, max size 100).

3.3.2 Endpoint Summary

Method	Endpoint	Role	Purpose	Req.
POST	/auth/register	Public	Register a student	FR-01
POST	/auth/login	Public	Log in, returns JWT + role	FR-02, FR-03
POST	/auth/logout	Any	Invalidate token	SR-04
POST	/auth/password-reset	Public	Request reset e-mail	FR-04
POST	/auth/password-reset/confirm	Public	Set new password with token	FR-04
GET/POST	/users	Admin	List / create users	FR-05
PUT/DELETE	/users/{id}	Admin	Update (role, status) / delete user	FR-05
GET/POST	/questions	Teacher	List / create questions	FR-07
PUT/DELETE	/questions/{id}	Teacher	Update / delete question	FR-07

Method	Endpoint	Role	Purpose	Req.
POST	/exams	Teacher	Create exam	FR-06, FR-08
PUT	/exams/{id}	Teacher	Update exam settings	FR-06, FR-20
POST	/exams/{id}/publish	Teacher	Publish/unpublish and assign groups	FR-09
GET	/exams/available	Student	List assigned exams (Upcoming/Available/Completed)	FR-10
POST	/exams/{id}/attempts	Student	Start attempt; returns questions (no keys) and endTime	FR-11, FR-12
PUT	/attempts/{id}/answers	Student	Autosave answers / review flags	FR-13, FR-14
POST	/attempts/{id}/submit	Student	Submit attempt	FR-15, FR-16
GET	/attempts/{id}/result	Student	Get score and status	FR-17
GET	/reports/exams/{id}	Teacher	Statistics; ?format=csv exports CSV	FR-18
GET	/reports/summary	Admin	System summary	FR-19
GET	/audit	Admin	Query audit log	FR-21, SR-08

3.3.3 Example: Login

Request — POST /api/v1/auth/login

```
{
  "email": "asha@test.com",
  "password": "Exam@2026"
}
```

Response 200 OK

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "expiresInSeconds": 1800,
  "role": "STUDENT",
  "name": "Asha K"
}
```

3.3.4 Example: Start Attempt

Request — POST /api/v1/exams/7cle.../attempts (header Authorization: Bearer <JWT>). **Response 201 Created**

```
{
  "attemptId": "a91f...",
  "endTime": "2026-10-05T10:35:00Z",
  "questions": [
    {
      "id": "q1", "type": "MCQ", "text": "2 + 2 = ?", "marks": 1,
      "options": [
        { "id": "o1", "text": "3" },
        { "id": "o2", "text": "4" },
        { "id": "o3", "text": "5" },
        { "id": "o4", "text": "6" }
      ]
    }
  ]
}
```

Note: no “correct” or “isCorrect” field is returned (SR-07).

3.3.5 Example: Autosave Answers

Request — PUT /api/v1/attempts/a91f.../answers. **Response 204 No Content** (or 409 if the attempt has ended).

```
{
  "answers": [
    { "questionId": "q1", "selectedOptions": ["o2"], "markedForReview": false },
    { "questionId": "q3", "selectedOptions": [], "markedForReview": true }
  ]
}
```

```
]
}
```

3.4 Error Handling

3.4.1 Strategy

- **Validate early:** all request DTOs are validated (Bean Validation) at the controller boundary; invalid input returns 400/422 before any business logic runs.
- **Central exception handling:** a global `@ControllerAdvice` maps domain exceptions to a uniform error response; stack traces and internal details are never sent to the client (SR-05, SR-10).
- **Fail safely:** on any authentication or authorization failure the request is denied and an audit entry is written; on evaluation failure the attempt stays SUBMITTED and the evaluation is retried (up to 3 times, then flagged for the Administrator).
- **Resilience:** database and e-mail calls have timeouts (5 s); the client retries autosave with back-off and keeps unsent answers locally until the network returns (NFR-07).
- **Observability:** each request has a correlation ID returned in the error body and written to the application log so that support staff can trace a failure.

3.4.2 Standard Error Response

```
{
  "timestamp": "2026-10-05T10:12:44Z",
  "status": 403,
  "code": "ACCESS_DENIED",
  "message": "You are not allowed to perform this action.",
  "path": "/api/v1/exams",
  "correlationId": "c0f3-9d1a"
}
```

3.4.3 Error Catalogue

Code	HTTP	Situation	Client behaviour
VALIDATION_FAILED	400 / 422	Missing/invalid field, weak password, duration out of range	Show message next to the field
INVALID_CREDENTIALS	401	Wrong e-mail or password (message does not reveal which)	Show generic error
ACCOUNT_LOCKED	423 / 401	5 failed attempts; locked for 15 minutes	Show remaining lock time
TOKEN_EXPIRED	401	JWT expired or blacklisted	Redirect to login
ACCESS_DENIED	403	Role not permitted / object not owned by caller	Show “not allowed” page
NOT_FOUND	404	Resource does not exist	Show not-found message
EXAM_NOT_AVAILABLE	409	Outside exam window or attempt limit reached	Show reason on exam card
ATTEMPT_CLOSED	409	Answer received after end time / already submitted	Show “time is over” and go to result
DUPLICATE_RESOURCE	409	E-mail already registered	Show message on e-mail field
RATE_LIMITED	429	Too many requests	Wait and retry (Retry-After header)
INTERNAL_ERROR	500	Unexpected server error	Show generic message with correlation ID; retry

3.5 User Interface Design

The user interface is a responsive single-page application. The main screens and their navigation are:

Screen	Role	Main elements
Login / Register / Reset	Public	Forms with inline validation and password-policy hints
Student Dashboard	Student	Tabs: Upcoming, Available, Completed; Start button enabled only in window
Exam Attempt	Student	Question panel, question palette (answered / review / unanswered), countdown timer, autosave indicator, Submit button with confirmation
Result	Student	Score, percentage, Pass/Fail, optional review
Teacher Dashboard / Question Bank / Exam Editor	Teacher	Tables with search, question form, exam settings form, publish control
Reports	Teacher, Admin	Statistics cards, result table, Export CSV
User Management / Audit Log	Admin	User table with role/status actions; filterable audit list

3.6 Design Decisions and Constraints

ID	Decision	Reason
DD-01	Stateless JWT authentication with Redis blacklist	Enables horizontal scaling (NFR-04) while still allowing logout (SR-04)
DD-02	Server-authoritative timer; client timer for display only	Prevents time tampering (SR-09)
DD-03	Answers autosaved to the database, not only in the browser	Recovery after connection loss (NFR-07)
DD-04	Evaluation runs in a separate service module with access to the answer key	Keeps the key away from the client and other modules (SR-07)
DD-05	Modular monolith instead of microservices	Simpler deployment and testing for the project team; modules can be split later
DD-06	Append-only audit table	Tamper evidence and accountability (SR-08)